LOCAL LLM TRANSLATION AND PARAPHRASING SYSTEM

Question 4 – Implementation

1. Question

Integrate a locally running Large Language Model with a Python application to perform text translation and paraphrasing.

2. Aim

To develop an interactive Streamlit application that uses a locally running Llama 3.2 Large Language Model through Ollama to translate text into selected languages and paraphrase text while preserving its original meaning.

3. Technologies Used

- Python 3.12.10
- Streamlit
- Ollama 0.33.2
- Llama 3.2 – local Large Language Model
- VS Code

4. Project Structure

Local_LLM_Text_Generation/

├── venv/ – Python virtual environment

├── app.py – Question 1 text generation

├── summarization.py – Question 2 text summarization

├── qa.py – Question 3 question answering

└── translation_paraphrasing.py – Question 4 translation and paraphrasing

5. Implementation Steps

1. Use the existing Local_LLM_Text_Generation project folder and virtual environment.
2. Reuse the installed Streamlit and Ollama Python packages.
3. Reuse the locally installed Llama 3.2 model through Ollama.
4. Create a Python file named translation_paraphrasing.py.
5. Design the Streamlit interface with local model selection and operation selection.
6. Provide Translation and Paraphrasing as the available operations.
7. For translation, allow the user to select a target language.
8. Accept the source text through a Streamlit text area.
9. Create an appropriate prompt based on the selected operation.
10. Send the prompt to Llama 3.2 through Ollama.
11. Display the translated or paraphrased result in the Streamlit interface.

6. Python Implementation

```
import streamlit as st
import ollama

st.set_page_config(
    page_title="Local LLM Translation and Paraphrasing",
    page_icon="🌐",
    layout="centered"
)

st.title("🌐 Local LLM Translation & Paraphrasing")

st.write(
    "Translate or paraphrase text using a locally running "
    "Large Language Model through Ollama."
)

model = st.selectbox(
    "Select Local LLM",
    ["llama3.2"]
)

operation = st.selectbox(
    "Select Operation",
    ["Translation", "Paraphrasing"]
)

if operation == "Translation":
    target_language = st.selectbox(
        "Select Target Language",
        ["English", "Tamil", "Telugu", "Hindi", "Malayalam"]
    )

text = st.text_area(
    "Enter your text",
    placeholder="Enter the text you want to process...",
    height=200
)

if st.button("Process Text", type="primary", width="stretch"):

    if not text.strip():
        st.warning("Please enter some text.")

    else:
        if operation == "Translation":
            prompt = f"""
You are a professional translation assistant.

Translate the following text into {target_language}.
```

```
Rules:
- Preserve the original meaning.
- Use natural and grammatically correct language.
- Do not add unnecessary information.
- Return only the translated text.

Text:
{text}
"""
        else:
            prompt = f"""
You are a professional paraphrasing assistant.

Rewrite the following text using different words
while keeping the original meaning.

Rules:
- Preserve the original meaning.
- Improve clarity and readability.
- Do not add new information.
- Do not remove important information.
- Return only the paraphrased text.

Text:
{text}
"""

        response = ollama.chat(
            model=model,
            messages=[
                {"role": "user", "content": prompt}
            ]
        )

        result = response["message"]["content"]

        st.subheader("Processed Text")
        st.write(result)
```

7. Working Principle

The application first allows the user to select the local LLM and choose either Translation or Paraphrasing. For translation, the user selects a target language. The entered text is then placed into an operation-specific prompt and sent to Llama 3.2 through Ollama. The generated result is returned to Python and displayed in the Streamlit interface.

8. System Workflow

12. User selects Llama 3.2.
13. User selects Translation or Paraphrasing.
14. For translation, the user selects the target language.
15. User enters the source text.

16. Python creates the appropriate prompt.
17. Ollama sends the prompt to the local Llama 3.2 model.
18. The model generates the translated or paraphrased text.
19. Streamlit displays the processed text.

9. Input

Translation test input: “Artificial Intelligence is transforming many areas of modern life. It is used in education, healthcare, banking and transportation.” The selected target language in the demonstrated run was Telugu.

10. Output

The application successfully generated Telugu text for the supplied English input. The displayed result preserves the main meaning of the original content while presenting it in the selected target language.

11. Paraphrasing Function

For paraphrasing, the same interface can be switched from Translation to Paraphrasing. The local LLM rewrites the source text using different wording while maintaining the original meaning and important information.

12. Application Output Screenshot

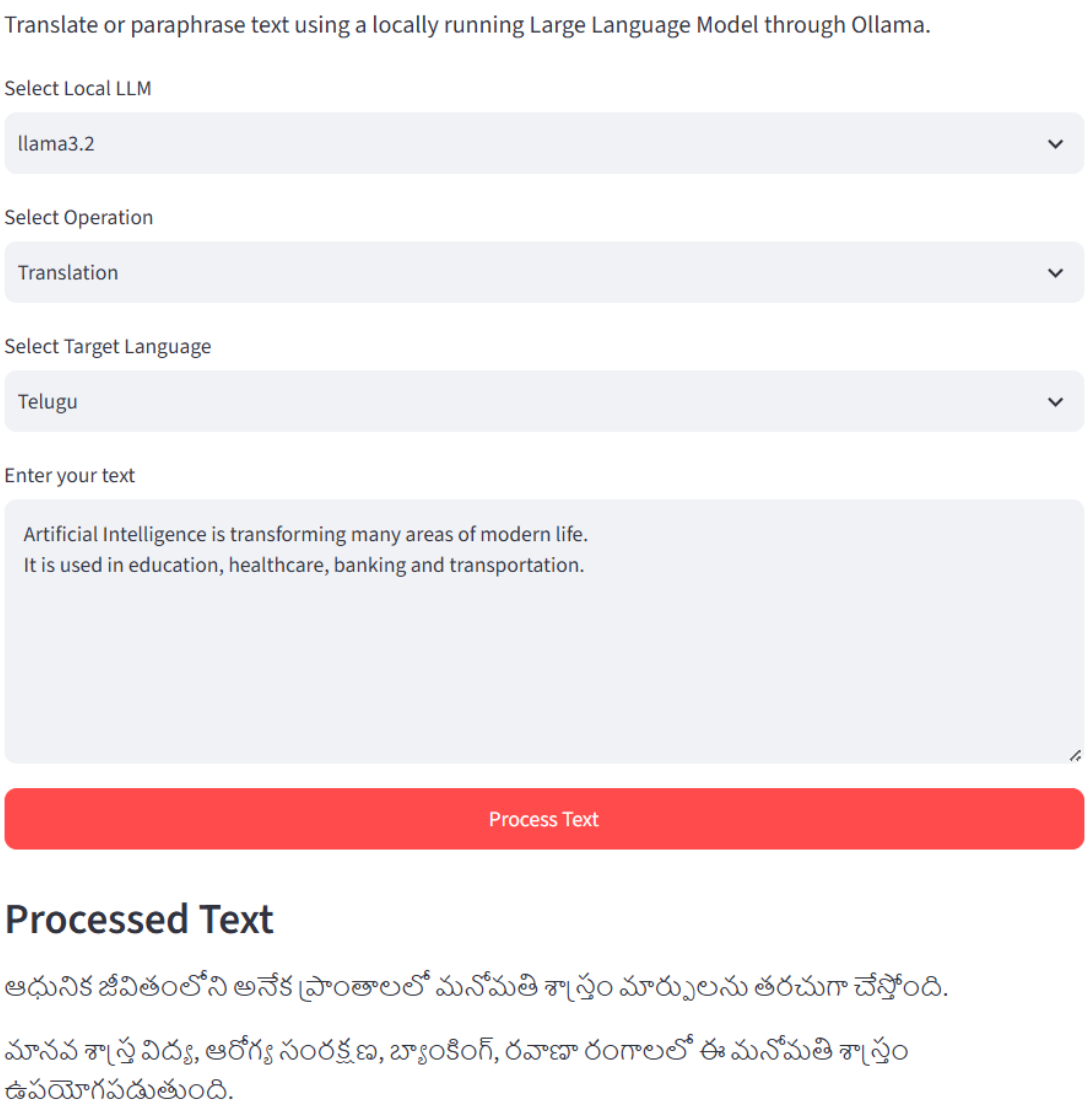

Figure 1: Streamlit interface showing Telugu translation using the local Llama 3.2 model

13. Result

Thus, a Python and Streamlit application was successfully integrated with a locally running Llama 3.2 Large Language Model through Ollama to perform text translation and paraphrasing. The demonstrated application successfully translated English text into Telugu.

14. Conclusion

The implementation demonstrates that a locally running Large Language Model can support multiple language-processing tasks through a single Python application. Ollama provides the local model interface, while Streamlit provides an interactive user interface for translation and paraphrasing.